

Multi-Stage Optimization by Prompting with Exploration and Recombination

Bui Van Hoa
22021163@vnu.edu.vn

Dinh Thi Phuong Thanh
22021167@vnu.edu.vn

Nguyen Huu Thang
22021154@vnu.edu.vn

Mai Hoang Bach
22021224@vnu.edu.vn

Abstract

In this work, we propose an improved method based on Optimization by Prompting (OPRO) called Evolutionary Optimization by Prompting (E-OPRO), a simple and effective method to leverage large language models (LLMs) as optimization tools, where the optimization task is described in natural language. We will expand the solution space by generating a series of new solutions, then perform selection and hybridization to create the best prompts. Then, using the LLM, we generate new solutions from prompts containing previously generated solutions along with their values; the new solutions are evaluated and added to the prompt for the next optimization step. Through experiments similar to the original method, we found that prompts generated by E-OPRO achieved superior scores. Code at https://github.com/bufihofa/OPRO_GEMINI8.

1 Introduction

Optimization is critical for all areas. Many optimization techniques are iterative: the optimization starts from an initial solution, then iteratively updates the solution to optimize the objective function (Amari, 1993; Qian, 1999; Kingma & Ba, 2015; Bäck & Schwefel, 1993; Rios & Sahinidis, 2013; Reeves, 1993). The optimization algorithm typically needs to be customized for an individual task to deal with the specific challenges posed by the decision space and the performance landscape, especially for derivative-free optimization.

In Large Language Models as Optimizers (Yang et al., 2024), they proposed Optimization by PROMpting (OPRO), a simple and effective approach to utilize large language models (LLMs) as optimizers. Instead of formally defining the optimization problem and deriving update steps with a programmed solver, they describe the optimization problem in natural language and instruct the LLM to iteratively generate new solutions based on the problem description and previously found solutions.

Although mixed linear models (LLMs) demonstrate the power of optimizing fundamental mathematical problems, OPROs still suffer from several limitations across all optimized LLMs that can hinder the ability to solve more complex problems. The main problem is that the optimal trajectory is easily biased and gets stuck at poor solutions. This is because OPROs inherently lack gradients, they lack a mechanism to control the search direction, and they rely heavily on meta-prompts. This makes the model susceptible to bias towards previous solutions and prone to getting stuck at suboptimal points.

To address this problem, we propose a method called Evolutionary Optimization by Prompting (E-OPRO). The main idea of this method consists of 3 stages:

1. Expanding the search space to develop diversity
2. Selecting and hybridizing high-quality solutions
3. Using OPRO as a deep optimization tool

We conducted evaluations on several LLMs, including *gemini-2.5-flash* and *gemini-2.5-flash-lite*. We optimized suggestions on GSM8K (Cobbe et al., 2021) and AQUA, following the experimental setup in OPRO.

2 The Proposed Method

2.1 Evolutionary Optimization by Prompting (E-OPRO)

E-OPRO is an improved strategy of the OPRO method from Google DeepMind, it has 3 main stage to optimize the performance of LLM. Instead of simply performing a single iterative process, E-OPRO split the solution finding process into specific steps in order to balance the discovery of various ideas and the exploitation of the best practices.

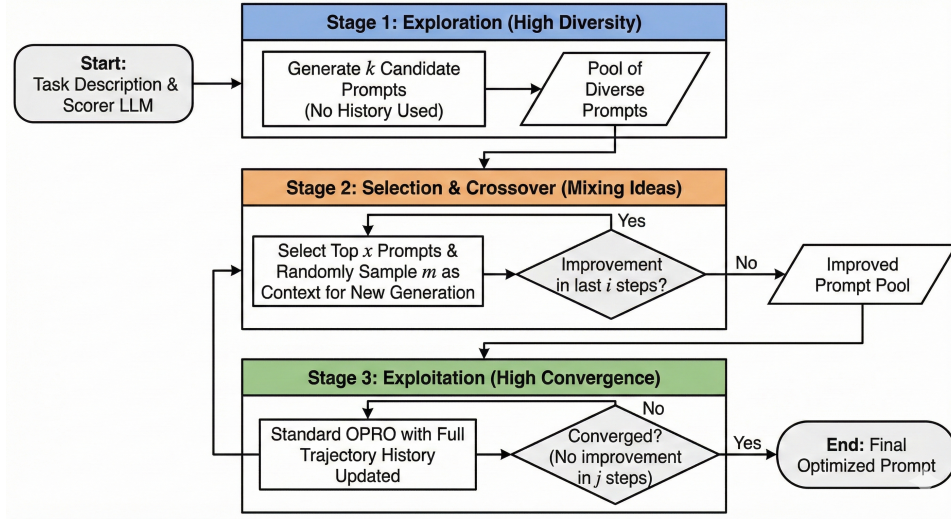


Figure 1: Overview of E-OPRO

2.2 Stage 1: Exploration

This stage acts as the foundation of the whole method, it focus on creating the richest and most various set of prompts possible. The primary goal of this stage is to prevent from getting stuck in local optimal from the start. By not limiting the model to a specific trajectory, OPRO can explore various solution that traditional method often skip.

In this stage, we run OPRO several times, each time it can generate k prompts without remembering the history so that it can not be influence by the old generated prompts. During the stage, the scores of the prompt usually unstable and highly random. We continue generate until we collect 60 prompts then we move to the next stage.

2.3 Stage 2: Selection and Crossover

After having a diverse prompt-set stage 1, stage 2 begins the process of refining and combining the best elements from the leading solutions. This stage aim to combine superior features from various prompts to create more optimize prompts. This is a crucial step to raise the overall quality of the prompt-set.

In order to start this stage, we choose top x prompts, which have the highest score, among 60 prompts that we collected from stage 1. We continuously run OPRO, but this time OPRO takes randomly m prompts in top x prompts to make a trajectory for generation. Although it generates a large amount of new prompts, we still do not update the main trajectory history. This method creates a random crossover between good ideas, allowing the model to experiment with different statement structures based on what has worked. After 10 steps without finding any improvement in the prompt score, we move to the final stage.

2.4 Stage 3: Exploitation

This is the last refinement stage to achieve the highest score. We still keep top x prompts from Stage 2, but this time, we update the Trajectory History, then we run the traditional OPRO. As a result, the scores converge to the highest and generate the final converged prompt. The Stage ends when there are no higher scores generated after 10 steps.

3 Experiments

We present the evaluation results for prompt optimization in this section. Our experiments demonstrate that E-OPRO brings a significant performance gain across the board.

3.1 Configuration

3.1.1 Model

In this research, the LLMs we use as the optimizer and the scorer are:

- Optimizer LLM: Pre-trained *gemini-2.5-flash*
- Scorer LLM: Pre-trained *gemini-2.5-flash-lite*

The reason why we choose these LLMs is quite simple. The *gemini-flash* and *gemini-flash-lite*, optimized for thinking logic with low latency and fast inference speed, are suited for OPRO’s multi-stage iterative workflow. Thanks to their fast processing ability, these models can generate and score

hundreds of prompts in a short time. In addition, *gemini-flash* is quite cheap, the multi-stage OPRO have to call the API thousands of time just to generate and score the prompt, *gemini-flash* is the ideal solution for us to make is work efficiency and balances the evaluation cost.

Gemini 2.5 is trained on large-scale multimodal datasets and is particularly strong in mathematics, which make them deeply adapt to understand the problem structure in GSM8K and AQUA, acting as an efficient optimizer. Moreover, the large context window allows to store the entire optimal trajectory in OPRO without losing historical data.

3.1.2 Benchmarks

Our primary benchmarks are GSM8K and AQUA. GSM8K is a benchmark of grade school math word problems with 7,473 training samples and 1,319 test samples, where chain-of-thought prompting (Wei et al., 2022) and the zero-shot instruction “Let’s think step by step.” (Kojima et al., 2022) have drastically improved the performance over the standard prompting. AQUA contains more complex algebra problems with multiple choice answer, it require a higher level of logical understanding and precise calculation skills.

For prompt optimization, we randomly sample 3.5% of the examples from the GSM8K training set. The same subset is used throughout the entire optimization process. This ensures that the task accuracies computed at intermediate optimization steps serve as approximations of the training accuracy on all 7,473 training examples. This design balances evaluation cost with generalization performance. Table 1 summarizes top instruction found on GSM8K with 2 different methods.

On AQUA, the optimization starts from an empty string as the initial instruction by default. For each task, we utilize a subset of 20% examples for prompt optimization, and the rest examples are for testing. Similar to GSM8K, we observe upward trends in optimization curves on almost all of AQUA tasks. As shown in Table 2, our method outperforms the baseline.

3.2 RQ1 - Hyperparameter Sensitivity

The parameters that we choose play a crucial part in determing the performance and cost of E-OPRO. For instance, we choose temperature $T = 1$ so that it can help the LLM to generate more unusual structures. If T is set

Method	Prompt	Accuracy
Baseline	We now determine the answer....	90.67%
	Let’s systematically and precisely compute the final answer....	92.27%
Optimized	Before finalizing your solution, review all calculations and logical steps taken. Then, re-read the original question one last time to confirm your final numerical answer precisely and completely addresses what was asked.	93.18%
	After solving, systematically verify your answer by re-reading the original question, confirming all conditions are met, re-evaluating each step for logical soundness, and optionally plugging the result back into the problem.	93.18%

Table 1: Test accuracies on GSM8K.

too low (e.g. $T = 0.2$), the model will only generate prompts idea around the original prompt. The number of generation at a time (k) should not be too high as well, after researching we found out that the higher k is (e.g $k = 20$), the bigger the chance of finding the solution, but the token cost will increase with the number of cores. Experiments show that $k = 8$ to $k = 12$ is the optimal number to balance between performance and cost. Another parameters are "x" and "m" (from Stage 2), we saw that if x is too small, the system will easily falls into premature convergence. Maintaining $x = 20$ and random crossover $m = 8$ helps maintain better prompts, allowing good but imperfect ideas to combine and shine at the last stage.

3.3 RQ2 - Why in stage 3, we use traditional OPRO instead of any other optimized method?

The reason of choosing Traditional OPRO for Stage 3 are not random, but a strategic calculation focused on optimizing the performance. After going through the Exploration and Crossover Stages, we have a high-quality amount of data. OPRO now acts as a filter, helping LLM identify which

Method	Prompt	Accuracy
Baseline	Let’s conclusively find the solution....	74.41%
	First, we will establish a clear framework for the problem....	73.62%
Optimized	After deriving a solution, confirm it against all given conditions and check for its presence among the provided options....	79.13%
	When multiple-choice options are provided, systematically evaluate each option, carefully breaking down the problem into appropriate cases or steps to efficiently test and eliminate incorrect answers....	78.74%

Table 2: Test accuracies on AQUA.

phrases are truly perfect for fine-tuning at the micro level.

Another reason is that, in stage 1 and 2, we have accumulated a list of high-quality prompts. So that, in stage 3, we do not need some dramatic changes or structure shifts. Traditional OPRO with the stable meta-prompts is the best tool for making small fine-tuning to reach the highest score. This model can identify successful patterns from history to make some adjustments, such as adding self-checking steps that we often overlook.

Evolutionary optimization methods usually maintain a certain stochastic variance to avoid local extrema. However, this variance can be a noise preventing the system to reach saturation. Traditional OPRO helps stabilize the meta-prompt, minimizing the risk of generating discrete and inconsistent prompts. This ensure that each loop in Stage 3 will directly improve the accuracy, rather than wasting resources on randomized trials.

4 Related Works

4.1 Prompt optimization

Prior works have developed soft prompt-tuning methods that optimize the prompt represented as task-specific continuous vectors, as well as performing discrete prompt optimization by gradient-guided search and reinforcement learning. These approaches become inapplicable when there is only API access to the LLM. Other works designed edit-based approaches for gradient-free prompt optimization, where the editing can be done with human defined operations (e.g., swapping two phrases) or language models (e.g., back translation) (Xu et al., 2022). Some recent works investigate LLMs for prompt optimization. Specifically, APE first uses the LLM to generate initial instructions. Afterwards, APE selects top instructions with the highest accuracies, then prompts the LLM with each individual instruction to generate a semantically similar variant of the initial instruction. APO in each step instructs the LLM to produce text feedback on how to update an old instruction. Different from edit-based approaches, the optimizer LLM in our work directly generates new instructions at each optimization step, and the optimizer LLM is merely asked to improve the task accuracy without being required to imitate past instructions. Compared to Zhou et al. (2022b) and Pryzant et al. (2023), our optimization process incorporates the past generated instructions with their scores in the meta-prompt, enabling the optimizer LLM to discover common patterns of high-quality instructions.

4.2 Prompting with natural language feedback

A recent line of work investigates approaches to improve the LLM performance by prompting with natural language feedback to revise the model output, which has shown effectiveness in reducing harmful LLM outputs, improving reasoning and code generation performance, dialogue applications (Nair et al., 2023; Madaan et al., 2023; Yuan et al., 2023), and so on (Kim et al., 2023; Wang et al., 2023). Specifically, Yuan et al. (2023) develops a human-in-the-loop framework for deriving system-level feedback from a collection of instance-level feedback, which is then used 20 Large Language Models as Optimizers for refining data. In our work, the optimizer LLM utilizes the optimization trajectory in the prompt, which implicitly requires the LLM to summarize the common characteristics among solutions with similar scores. We consider incorporating explicit natural language feedback on

generated solutions for later optimization steps as future work.

4.3 Tuning language models for optimization

Some previous works tune or prompt language models to behave as mutation and crossover operators in evolutionary algorithms. Meyerson et al. (2023) utilizes language models with few-shot exemplars to propose evolutionary cross-overs on tasks such as image and code generation. In Lehman et al. (2022), the large language model trained on code diff generation is used as the mutation operator, and they further design a fine-tuning method to improve performance in the Sodarace domain for robot simulation. Evo-Prompting (Chen et al., 2023a) uses large language models to evolve neural network architectures, where they combine evolutionary search with soft prompt tuning. With respect to taking the trajectory as the input for optimization, OptFormer (Chen et al., 2022) trains a transformer model on large collections of hyperparameter optimization data. On the other hand, our work performs optimization solely by prompting without additional training.

5 Conclusion

This research has successfully proposed the E-OPRO method, a multi-stage optimization aimed to tackle the limitations of diversity and non-convergence of the OPRO system. By analyzing the optimization process into three specific stages: Exploration, Crossover and Exploitation. The core idea of E-OPRO allow the system to maximize its search space before focusing on fine-tuning the instruction to achieve optimal performance, which is often overlooked in single-stage processes.

The efficiency of E-OPRO were proved through tests on dataset of mathematical benchmarks. The results showed us that the method achieved 93.18% accuracy on GSM8K and significantly improved by 4.72% on the AQUA dataset (reaching 79.13%) compared to the traditional OPRO. The study of convergence also confirm about the role of the Stage 1 in establishing the foundation for breakthroughs in accuracy. This result demonstrates the efficiency of E-OPRO in optimizing prompt.

However, the research still remains some limited by the cost of token during the method. In the future, the research will focus on automating control the parameter (such as the number of steps, samples, v.v) to optimize operating

cost. At the same time, the method will be expanded to include multimodal tasks and more complex setups to confirm the generality of the multi-stage optimization strategy in larger-scale problems.

References

- [1] C. Yang, X. Wang, Y. Lu, H. Liu, Q. V. Le, D. Zhou, and X. Chen, “Large language models as optimizers,” *arXiv preprint arXiv:2309.03409v3*, 2024.
- [2] J. Wei, X. Wang, D. Schuurmans, M. Maeda, F. Zhao, Y. Xia, E. Chi, Q. V. Le, and D. Zhou, “Chain-of-thought prompting elicits reasoning in large language models,” *Advances in Neural Information Processing Systems*, vol. 35, pp. 24824–24837, 2022.
- [3] Y. Zhou, A. I. Madaan, P. Liu, S. K. Gupta, R. Mushkindan, S. Reddy, N. Neubig, and G. Neubig, “Large language models are human-level prompt engineers,” *arXiv preprint arXiv:2211.01910*, 2022.
- [4] K. Cobbe, V. Kosaraju, M. Bavarian, M. Chen, H. Jun, L. Kaiser, M. Plappert, J. Tworek, J. Hilton, and R. Nakano, “Training verifiers to solve math word problems,” *arXiv preprint arXiv:2110.14168*, 2021.
- [5] W. Ling, D. Yogatama, C. Dyer, and P. Blunsom, “Program induction by rationale generation: Learning to solve algebraic word problems,” *arXiv preprint arXiv:1705.04148*, 2017.
- [6] R. Anil *et al.*, “PaLM 2 technical report,” *arXiv preprint arXiv:2305.10403*, 2023.
- [7] S. Kojima, S. S. Gu, M. Reid, Y. Matsuo, and Y. Iwasawa, “Large language models are zero-shot reasoners,” *Advances in Neural Information Processing Systems*, vol. 35, pp. 22199–22213, 2022.
- [8] H. Liu, C. Yang, X. Chen, Q. V. Le, D. Zhou, and X. Wang, “Evo-Prompt: Enforcing evolutionary search in prompt optimization,” *arXiv preprint arXiv:2309.08532*, 2023.
- [9] T. Brown *et al.*, “Language models are few-shot learners,” *Advances in Neural Information Processing Systems*, vol. 33, pp. 1877–1901, 2020.

- [10] S. Amari, “Backpropagation and stochastic gradient descent method,” *Neurocomputing*, vol. 5, no. 4–5, pp. 185–196, 1993.
- [11] D. P. Kingma and J. Ba, “Adam: A method for stochastic optimization,” in *Proc. Int. Conf. Learning Representations (ICLR)*, 2015.
- [12] X. Liu, Y. Zheng, Z. Du, M. Ding, Y. Qian, Z. Yang, and J. Tang, “GPT understands, too,” *arXiv preprint arXiv:2103.10385*, 2021.
- [13] T. Bäck and H.-P. Schwefel, “An overview of evolutionary algorithms for parameter optimization,” *Evolutionary Computation*, vol. 1, no. 1, pp. 1–23, 1993.
- [14] L. M. Rios and N. V. Sahinidis, “Derivative-free optimization: A review of algorithms and comparison of software implementations,” *Journal of Global Optimization*, vol. 56, pp. 1247–1293, 2013.
- [15] C. R. Reeves, *Modern Heuristic Techniques for Combinatorial Problems*. New York, NY, USA: John Wiley & Sons, 1993.
- [16] B. Lester, R. Al-Rfou, and N. Constant, “The power of scale for parameter-efficient prompt tuning,” *arXiv preprint arXiv:2104.08691*, 2021.
- [17] X. L. Li and P. Liang, “Prefix-tuning: Optimizing continuous prompts for generation,” *arXiv preprint arXiv:2101.00190*, 2021.
- [18] G. Qin and J. Eisner, “Learning how to ask: Querying language models with mixtures of soft prompts,” *arXiv preprint arXiv:2104.06599*, 2021.
- [19] T. Shin, Y. Razeghi, R. L. Logan IV, E. Wallace, and S. Singh, “Auto-Prompt: Eliciting knowledge from language models with automatically generated prompts,” *arXiv preprint arXiv:2010.15980*, 2020.